

# PROJECT REPORT

## Blog Platform – MERN Stack Application

Submitted by: Komal Joshi

---

### 1. INTRODUCTION

#### 1.1 Project Overview

The Blog Platform is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The application allows users to register, log in securely, and perform CRUD (Create, Read, Update, Delete) operations on blog posts.

The platform ensures secure authentication and authorization so that users can only modify their own posts. It provides a responsive and user-friendly interface for seamless interaction.

---

#### 1.2 Objectives

- To implement secure user authentication using JWT.
  - To enable full CRUD functionality for blog posts.
  - To ensure users can only edit or delete their own posts.
  - To design a responsive and user-friendly interface.
  - To integrate frontend and backend using REST APIs.
- 

### 2. TECHNOLOGY STACK

#### Frontend

- React.js
- React Router
- Context API
- Axios
- CSS3

#### Backend

- Node.js
- Express.js
- JSON Web Token (JWT)
- bcryptjs

#### Database

- MongoDB
- Mongoose ODM

## Tools Used

- Postman
  - Git
  - Visual Studio Code
- 

## 3. SYSTEM ARCHITECTURE

The system follows a client-server architecture:

Client (React – Port 3000)

↓

API Calls using Axios (JWT attached in headers)

↓

Server (Express – Port 5000)

↓

Middleware (Authentication & Error Handling)

↓

Routes (Authentication Routes & Post Routes)

↓

Controllers (Business Logic)

↓

Database (MongoDB)

---

## 4. KEY FEATURES IMPLEMENTED

### 4.1 Authentication Module

- User registration with proper validation.
  - Secure login using JWT tokens.
  - Password hashing using bcrypt before storing in database.
  - Persistent login using localStorage.
  - Protected routes accessible only to authenticated users.
- 

### 4.2 Post Management Module

- Create new blog posts (only for authenticated users).
  - View all posts (public access).
  - View single post details.
  - Edit own posts.
  - Delete own posts.
  - View personal posts in user profile.
- 

### 4.3 User Interface

- Responsive design for different screen sizes.

- Loading states during API requests.
  - Toast notifications for success and error messages.
  - Client-side form validation.
  - Proper error handling and user feedback.
- 

## 5. DATABASE SCHEMA

### User Collection

- username (String, required, unique)
  - email (String, required, unique)
  - password (String, required)
  - createdAt (Date)
  - updatedAt (Date)
- 

### Post Collection

- title (String, required)
  - content (String, required)
  - user (ObjectId, reference to User)
  - createdAt (Date)
  - updatedAt (Date)
- 

## 6. API ENDPOINTS

### Authentication Routes (/api/auth)

- POST /register – Register a new user
- POST /login – User login
- GET /me – Get current logged-in user

### Post Routes (/api/posts)

- GET / – Get all posts
  - GET /:id – Get a single post
  - POST / – Create a new post
  - PUT /:id – Update a post
  - DELETE /:id – Delete a post
  - GET /user/profile – Get posts of logged-in user
- 

## 7. IMPLEMENTATION HIGHLIGHTS

### 7.1 Authentication Flow

- User registers → Password is hashed using bcrypt.
- User logs in → JWT token is generated.
- Token is stored in localStorage.

- Axios interceptor attaches token to all protected API requests.
  - Backend middleware verifies token before granting access.
- 

## 7.2 Authorization

- Before updating or deleting a post, ownership is verified.
  - Only the user who created the post can modify it.
  - Invalid or expired tokens redirect the user to login page.
- 

## 7.3 State Management

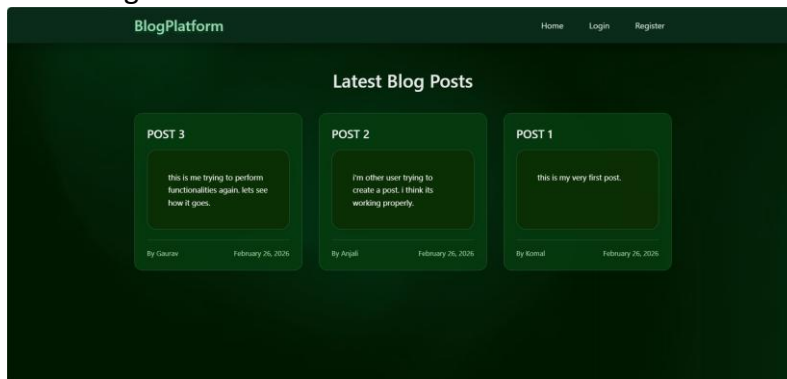
- Context API is used for global authentication state.
  - Local component state handles form inputs and UI states.
  - Loading indicators are used for asynchronous operations.
- 

## 8. CHALLENGES FACED AND SOLUTIONS

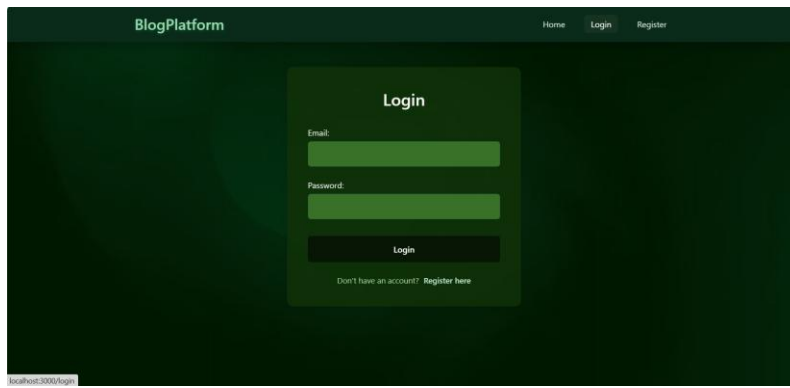
1. JWT Token Persistence
    - Solution: Store token in localStorage and fetch user data on page refresh.
  2. Post Ownership Validation
    - Solution: Compare logged-in user ID with post owner ID before updates or deletion.
  3. CORS Issues
    - Solution: Enable CORS middleware in Express backend.
  4. State Management
    - Solution: Implement Context API for managing global authentication state.
  5. Form Validation
    - Solution: Apply both client-side and server-side validation.
- 

## 9. SCREENSHOTS

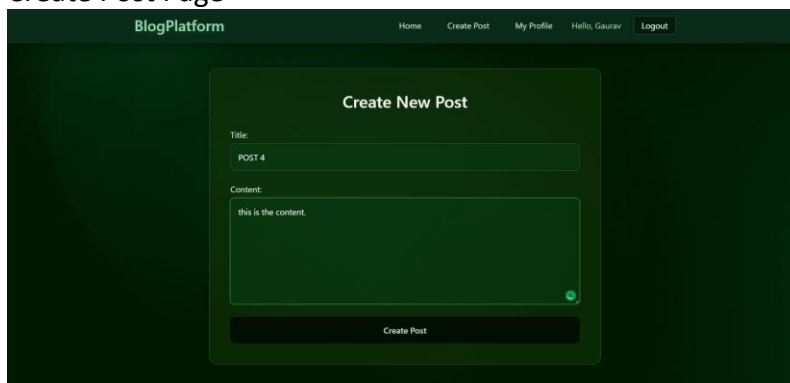
### Home Page



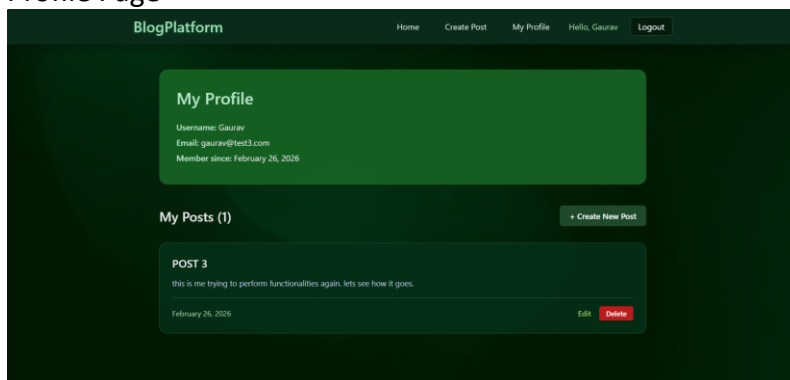
### Login Page



## Create Post Page



## Profile Page



---

## 12. CONCLUSION

The Blog Platform successfully demonstrates a complete MERN stack application with secure authentication and full CRUD functionality.

All major objectives were achieved:

- User authentication system
- Post creation, reading, updating, and deletion
- User-specific content management
- Responsive and intuitive user interface
- Proper validation and error handling

This project provided practical experience in full-stack development and strengthened understanding of authentication, authorization, API integration, and database design.

---

Name: Komal Joshi

Date: 26.02.2026

P.S. I tried green theme as I wanted to experiment, if it is not professional and need changes I'll definitely try to improve. Thankyou :)